

Soft Delete in Spring Boot API

Why Use PATCH for Soft Delete?

In hard delete, we permanently remove the row from the database.

Example:

```
DELETE /api/students/1
```

But in soft delete, we do not actually delete the row.

Instead, we only update one field:

```
isDeleted = true
```

So the student record still remains in the database, but it is marked as deleted.

Because we are only changing one specific field, soft delete is technically a **partial update**.

That is why using `PATCH` makes sense for soft delete.

```
PATCH /api/students/1/soft-delete
```

PATCH vs DELETE

Operation	What happens?	HTTP Method
Hard Delete	Row is permanently removed from the database	<code>DELETE</code>
Soft Delete	Row remains in the database, but <code>isDeleted</code> becomes <code>true</code>	<code>PATCH</code>

So for hard delete:

```
DELETE /api/students/1
```

For soft delete:

```
PATCH /api/students/1/soft-delete
```

Creating a Custom Finder Method for Soft Delete

When we use soft delete, our table contains a column like this:

```
is_deleted
```

In the Java entity class, we can represent it like this:

```
@Column(name = "is_deleted")  
private Boolean isDeleted = false;
```

Here, the default value is `false`, which means the student is active and not deleted.

When a student is soft deleted, we update this value to:

```
isDeleted = true;
```

Problem with Normal Fetch API

Normally, when we fetch all students, we may use:

```
studentRepository.findAll();
```

But there is a problem.

`findAll()` fetches every row from the database, including soft

For example:

id	name	is_deleted
1	Rahul	false
2	Priya	true
3	Aman	false

If we use `findAll()`, it may return all three students.

But in normal application behavior, soft-deleted students should not be shown.

So we only want records where:

```
isDeleted = false
```

In SQL, the query would look like this:

```
SELECT * FROM students  
WHERE is_deleted = false;
```

Using Spring Data JPA Derived Query Method

Spring Data JPA allows us to create queries by simply writing method names.

Instead of writing SQL manually, we can create this method inside the repository:

```
List<Student> findByIsDeletedFalse();
```

Spring Data JPA reads the method name and automatically generates the required query.

This method means:

```
Find all students where isDeleted is false.
```

Equivalent SQL idea:

```
SELECT * FROM students
WHERE is_deleted = false;
```

This type of method is called a **derived query method**.

How `findByIsDeletedFalse()` Works

Method:

```
List<Student> findByIsDeletedFalse();
```

Breakdown:

Part	Meaning
<code>findBy</code>	Query starts here
<code>IsDeleted</code>	Entity field name
<code>False</code>	Condition should be false

So Spring understands the method as:

```
Find students where isDeleted is false.
```

This works because Spring Data JPA follows naming conventions. It reads the method name, identifies the field, understands the condition, and creates the query automatically.

Why Add `findByIdAndIsDeletedFalse()` ?

For getting a student by ID, we may normally write:

```
studentRepository.findById(id);
```

But this can create an issue when soft delete is used.

Suppose a student exists in the database but is already soft

id	name	is_deleted
1	Rahul	true

If we use:

```
findById(1L)
```

it will still return Rahul because the row still exists in the database.

But from the application's point of view, a soft-deleted student should behave like an inactive or hidden record.

So for normal get-by-id APIs, we should fetch only students that are not deleted.

For that, we can create this repository method:

```
Optional<Student> findByIdAndIsDeletedFalse(Long id);
```

This means:

```
Find student where id = given id AND isDeleted = false.
```

Equivalent SQL idea:

```
SELECT * FROM students  
WHERE id = 1  
AND is_deleted = false;
```

Breakdown of

`findByIdAndIsDeletedFalse()`

Method:

```
Optional<Student> findByIdAndIsDeletedFalse(Long id);
```

Breakdown:

Part	Meaning
<code>findBy</code>	Query starts here
<code>Id</code>	Match the student ID
<code>And</code>	Add another condition
<code>IsDeleted</code>	Check the <code>isDeleted</code> field
<code>False</code>	Value should be false

So Spring Data JPA understands:

`Find` a student where `id` matches and `isDeleted` is `false`.

Final Repository Methods

Inside the repository interface, we can add:

```
public interface StudentRepository extends JpaRepository<Student, Long> {  
  
    List<Student> findByIsDeletedFalse();  
  
    Optional<Student> findByIdAndIsDeletedFalse(Long id);  
}
```

Now we can use:

```
findByIsDeletedFalse()
```

to fetch all active students.

And:

```
findByIdAndIsDeletedFalse(id)
```

to fetch one active student by ID.

Final Flow

For Soft Delete

Instead of deleting the row permanently:

```
studentRepository.deleteById(id);
```

we first fetch the student, then update the field:

```
student.setIsDeleted(true);  
studentRepository.save(student);
```

So the row remains in the database, but it becomes hidden from normal APIs.

Key Takeaways

- Soft delete means marking a record as deleted instead of removing it permanently.
- In soft delete, we usually update a field like `isDeleted`.
- Since only one field is updated, `PATCH` is a suitable HTTP method.
- `DELETE` is suitable when the row is actually removed from the database.
- `findAll()` may return soft-deleted records also.
- To fetch only active records, we can use custom finder methods.
- Spring Data JPA can generate queries automatically from method names.
- Methods like `findByIsDeletedFalse()` and `findByIdAndIsDeletedFalse()` are called derived query methods.
- Soft-deleted records should usually be hidden from normal get APIs.